Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого
—
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 3

«Разработка, интеграция, тестирование и развертывание сложного программного обеспечения. Часть 1»

по дисциплине «Технологии разработки современного программного обеспечения»

Выполнили
студент гр.5151003/30801 Трифанов А.С.
<подпись>

студент гр.5151003/30801 Игнатов Д.П.
<подпись>

Преподаватель
Ассистент преподавателя Проурзин О.В.
<подпись>

Санкт-Петербург
2026

1. Цель работы

Целью работы – получение практических навыков анализа сложного программного обеспечения для интеграции дополнительной бизнес-логики, а также теоретические знания о построения конвейера в рамках CI/CD.

2. Задачи

В рамках выполнения лабораторной работы необходимо:

1. Выбрать один из предложенных образовательных проектов;
2. Выбрать одну из тем, предложенных для развития данного образовательного проекта;
3. Сформировать команду, размер которой определяется в приложении согласно выбранной теме;
4. Решить задачи, указанные в приложении для выбранной темы.

3. Ход работы

3.1. Анализ образовательного проекта

В качестве объекта исследования и практической реализации в рамках данной лабораторной работы был выбран образовательный проект LabServer. Данный проект представляет собой современную многокомпонентную распределенную систему, архитектура которой логически разделена на следующие ключевые уровни:

— **Серверная часть (Backend):**

Реализована в виде RESTful API на базе кроссплатформенного фреймворка ASP.NET Core. Данный компонент является ядром системы, отвечающим за маршрутизацию запросов, обработку бизнес-логики, интеграцию с внешними сервисами и взаимодействие с базой данных.

— **Клиентская часть (Frontend):**

Представляет собой два независимых веб-приложения (Single Page Applications), разработанных с использованием фреймворка Angular. Система разделена на «Dashboard» (пользовательский интерфейс для студентов) и «Admin» (панель администратора для

управления учебным процессом), что позволяет разделить контексты безопасности и снизить нагрузку на клиентские устройства.

— **Уровень хранения данных (Storage):**

Для обеспечения надежного хранения персистентных данных (информации о пользователях, курсах и результатах лабораторных работ) используется реляционная СУБД PostgreSQL.

Подобный технологический стек и многозвенная архитектура обуславливают необходимость применения современных DevOps-подходов к развертыванию. Управление запуском нескольких независимых сервисов, контроль их состояния и обеспечение безопасной сетевой связности между ними (например, доступ бэкенда к закрытой базе данных) требуют использования системы оркестрации контейнеров.

В связи с этим для развертывания проекта выбрана платформа Kubernetes. Данный оркестратор позволяет не только автоматизировать процесс доставки компонентов системы в изолированных Docker-контейнерах, но и предоставляет встроенные механизмы для управления их жизненным циклом, маршрутизации внутреннего трафика и автоматического восстановления при программных сбоях, что является фундаментальным требованием для последующей настройки системы мониторинга.

3.2. Анализ типовых конфигураций для развертывания многокомпонентного ПО в кластере Kubernetes

Развертывание многокомпонентного программного обеспечения, такого как LabServer, в среде оркестратора Kubernetes требует применения специализированных абстракций (ресурсов). Поскольку базовые вычислительные единицы кластера — поды (Pods) — являются непостоянными сущностями, их прямое использование не обеспечивает отказоустойчивости.

В ходе анализа архитектуры проекта были выделены следующие типовые конфигурации и объекты Kubernetes, необходимые для построения надежной инфраструктуры:

1. **Контроллер развертывания (Deployment)**

Для управления жизненным циклом каждого компонента системы (Backend, Frontend, PostgreSQL) используется объект Deployment. Данный ресурс предоставляет механизмы декларативного управления подами и обеспечивает:

— Обеспечение отказоустойчивости:

Контроллер непрерывно отслеживает состояние кластера и сравнивает его с желаемым (заданным в манифесте). Если контейнер завершается с ошибкой или удаляется, Deployment автоматически создает новый под, что гарантирует постоянную доступность сервиса.

— Управление репликацией:

Позволяет горизонтально масштабировать систему, задавая необходимое количество идентичных подов (replicas), распределяя тем самым нагрузку.

— Контроль конфигурации:

Внутри ресурса Deployment определяются параметры запуска контейнеров: используемые Docker-образы (image), открытые порты (containerPort), а также переменные окружения (env).

2. Маршрутизация и сетевая связность (Service)

Поскольку поды могут динамически пересоздаваться с присвоением новых IP-адресов, для обеспечения стабильного сетевого взаимодействия между компонентами LabServer применяется абстракция Service. В проекте используются два типа сервисов:

— ClusterIP (внутренняя маршрутизация):

Типовая конфигурация для скрытых от конечного пользователя компонентов. В нашей архитектуре применяется для базы данных PostgreSQL (db-service). Это позволяет бэкенду обращаться к базе данных по статичному внутреннему DNS-имени, независимо от того, на каком физическом узле запущен под с БД.

— NodePort (внешний доступ):

Применяется для клиентской части приложения. Данный тип сервиса открывает определенный порт на всех узлах кластера и пробрасывает внешний HTTP-трафик внутрь кластера на порт контейнера (4200), позволяя студентам взаимодействовать с системой через веб-браузер.

3. **Интеграция конфигураций (Environment Configuration)**

Развертывание многокомпонентной системы требует соблюдения принципов конфигурации "Twelve-Factor App". Передача параметров (имя базы данных, логины, пароли) реализована через декларативное описание переменных окружения непосредственно в манифестах Deployment. Это позволяет разделить конфигурацию и код, обеспечивая гибкость при переносе проекта из среды разработки в производственную среду (Production)

Результаты и вывод по разделу:

Практическая реализация описанных абстракций (Deployment и Service) выполнена в виде конфигурационных манифестов: db.yaml для развертывания базы данных и app.yaml для развертывания серверной и клиентской частей системы. Исходный код разработанных файлов представлен в разделе "Приложения"

Анализ показал, что применение связки манифестов Deployment и Service формирует отказоустойчивую типовую конфигурацию. Deployment гарантирует, что приложения всегда запущены, а Service выступает в роли встроенного балансировщика нагрузки и DNS-резолвера, связывая фронтенд, бэкенд и базу данных в единую изолированную, но масштабируемую систему. Данная архитектура является оптимальной базой для внедрения подсистем мониторинга и автоматических проверок живости.

3.3. Проектирование системы мониторинга живучести (Liveness Probes)

В соответствии с заданием темы №17, критически важным этапом развертывания является настройка подсистемы контроля состояния сервисов. В распределенных системах контейнер может формально продолжать работу (процесс с PID 1 не завершен), однако само приложение может находиться в состоянии взаимоблокировки, потерять соединение с базой данных или уйти в бесконечный цикл.

Для выявления таких ситуаций и обеспечения высокой доступности системы была спроектирована подсистема мониторинга живучести на базе встроенного механизма

Kubernetes — Liveness Probes (пробы живости). Данный механизм реализуется агентом kubelet на каждом узле кластера.

В ходе проектирования архитектуры проверок были проанализированы особенности работы используемых фреймворков (ASP.NET Core и Angular). Было принято решение отказаться от стандартных HTTP-проверок (HTTP GET на эндпоинт /health) в пользу проверок доступности сетевого порта на уровне транспортного уровня модели OSI (tcpSocket). Этот подход позволяет гарантированно отслеживать готовность демона внутри контейнера принимать входящие TCP-соединения без необходимости модификации исходного кода учебного проекта.

Для компонентов системы были спроектированы следующие индивидуальные правила мониторинга:

3.3.1. Правила мониторинга для серверной части (Backend)

— Метод проверки — tcpSocket на порту 5288.

— Задержка инициализации (initialDelaySeconds) — 60 секунд.

Данный параметр был рассчитан эмпирическим путем. Значительная задержка перед первой проверкой обусловлена асинхронной природой запуска многокомпонентной среды: бэкенду необходимо дождаться полного запуска контейнера СУБД PostgreSQL, после чего инициализировать подключения к базе данных и запустить ядро ASP.NET.

— Период опроса (periodSeconds) — 15 секунд.

Оптимальный баланс между скоростью реакции на сбой и отсутствием излишней сетевой нагрузки на kubelet.

Была выполнена проверка настройки Liveness Probe для серверной части Backend (Рисунок 3.1):

```
PS C:\dev\devops-lab-17> kubectl describe pod -l app=backend
Name:             backend-7dbd976b69-zsc6v
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Sat, 25 Apr 2026 22:23:30 +0300
Labels:           app=backend
                  pod-template-hash=7dbd976b69
Annotations:      <none>
Status:           Running
IP:               10.244.0.3
IPs:
  IP:           10.244.0.3
Controlled By:  ReplicaSet/backend-7dbd976b69
Containers:
  backend:
    Container ID:   docker://858e72b1aec58ccdcb967b5644157ac054812e0be9d10dabade16fdd10f69753
    Image:          trialextime/labserver-backend:v1
    Image ID:       docker-pullable://trialextime/labserver-backend@sha256:b96f14df2a4a232d8132ecb1dcc651b2c37a53decdafa9e174ef9d242f0
9a882
    Port:           5288/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Sat, 25 Apr 2026 22:23:44 +0300
    Ready:          True
    Restart Count:  0
    Liveness:       tcp-socket :5288 delay=60s timeout=1s period=15s #success=1 #failure=3
    Environment:
      ConnectionStrings__PgSQL:  Host=db-service;Port=5432;Database=labs;Username=postgres;Password=changeit
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-b5hwc (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-b5hwc:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:                      <none>
```

Рисунок 3.1 - Проверка настройки Liveness Probe для серверной части Backend

3.3.2. Правила мониторинга для клиентской части (Frontend / Dashboard)

— Метод проверки — tcpSocket на порту 4200.

— Задержка инициализации (initialDelaySeconds) — 120 секунд.

Использование режима разработчика Angular (ng serve) внутри контейнера подразумевает длительный процесс горячей компиляции TypeScript-кода и сборки модулей Webpack при старте. Если не задать высокую задержку инициализации, kubelet убьет контейнер за отсутствие ответа еще до того, как фронтенд успеет скомпилироваться. Ожидание в 2 минуты предотвращает попадание пода в цикл бесконечных перезапусков (CrashLoopBackOff).

— Период опроса (periodSeconds) — 15 секунд.

Была выполнена проверка настройки Liveness Probe для клиентской части Dashboard (Рисунок 3.2):

```
PS C:\dev\devops-lab-17> kubectl describe pod -l app=dashboard
Name:             dashboard-5f478c95f7-z8jvx
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Sat, 25 Apr 2026 22:23:30 +0300
Labels:           app=dashboard
                  pod-template-hash=5f478c95f7
Annotations:      <none>
Status:           Running
IP:               10.244.0.4
IPs:
  IP:           10.244.0.4
Controlled By:  ReplicaSet/dashboard-5f478c95f7
Containers:
  dashboard:
    Container ID:   docker://9433725be828e39f0f903e157ec1035fc5fd12f42d2ce2fccfa56ecb915ea5d3
    Image:          trialextime/labserver-dashboard:v1
    Image ID:       docker-pullable://trialextime/labserver-dashboard@sha256:9a664e6115efdab0247f53da94644f828c38fc20ca0e336764e0e806e
6601bb6
    Port:           4200/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Sat, 25 Apr 2026 22:24:37 +0300
    Ready:          True
    Restart Count:  0
    Liveness:       tcp-socket :4200 delay=120s timeout=1s period=15s #success=1 #failure=3
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-45vhd (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-45vhd:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:                      <none>
```

Рисунок 3.2 — Проверка настройки Liveness Probe для клиентской части Dashboard

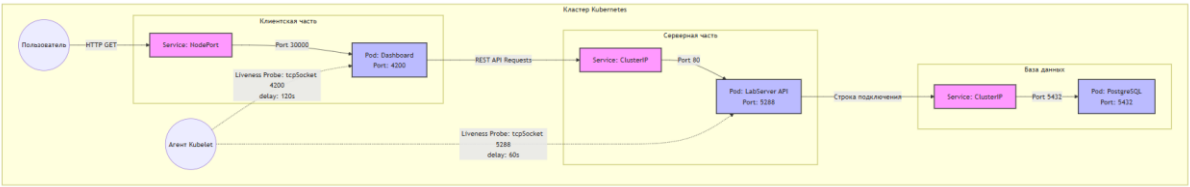

Рисунок 3.3 — UML-диаграмма развертывания компонентов и механизма Liveness Probes

(Исходный файл рисунка находится в репозитории рядом с отчетом)

```
PS C:\dev\devops-lab-17> kubectl get pods
NAME                         READY   STATUS    RESTARTS   AGE
backend-7dbd976b69-zsc6v     1/1     Running   0          105m
dashboard-5f478c95f7-z8jvx   1/1     Running   0          105m
postgres-849fdff8d4-7qzn2    1/1     Running   0          105m
```

Рисунок 3.4 — Состояние подов проекта LabServer после развертывания в Kubernetes

```
PS C:\dev\devops-lab-17> kubectl get svc
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
backend-service     ClusterIP   10.98.116.208   <none>        80/TCP         106m
dashboard-service   NodePort    10.107.175.3    <none>        80:30000/TCP   106m
db-service          ClusterIP   10.108.52.82    <none>        5432/TCP       106m
kubernetes          ClusterIP   10.96.0.1       <none>        443/TCP        107m
```

Рисунок 3.5 — Список Kubernetes Service для компонентов проекта LabServer

Логика реакции на инциденты:

Если в ходе эксплуатации системы порт сервиса перестает отвечать на запросы kubelet (превышен таймаут или исчерпан лимит неудачных попыток — по умолчанию 3 попытки), Kubernetes помечает контейнер как Unhealthy. После этого контроллер отправляет сигнал SIGTERM для мягкого завершения процесса, а затем принудительно перезапускает контейнер.

Таким образом, спроектированная подсистема реализует парадигму автоматического самовосстановления системы. Статистика срабатывания данных правил, количество перезапусков и общие статусы жизненного цикла подов (Running, Failed, Pending) передаются по метрике kube_pod_status_phase в систему внешнего мониторинга Prometheus для последующей агрегации и визуализации на стороне Студента Б.

3.4. Выбор инструментов мониторинга

Для внешнего мониторинга состояния развернутого Kubernetes-кластера были рассмотрены два подхода: Prometheus совместно с Grafana и Zabbix. Оба инструмента позволяют собирать метрики и отображать состояние сервисов, однако для cloud-native инфраструктуры на базе Kubernetes они отличаются по удобству интеграции и модели эксплуатации.

Таблица 3.1 — Сравнительный анализ инструментов мониторинга

Критерий	Prometheus + Grafana	Zabbix
Архитектура сбора метрик	Pull-модель: Prometheus сам опрашивает HTTP-endpoints и собирает time-series метрики.	Agent-based подход: данные собираются через агенты,

		proxy и checks; возможны active/passive checks.
Интеграция с Kubernetes	Нативная интеграция: service discovery, kube-state-metrics, node-exporter, ServiceMonitor/PodMonitor, Helm-чарты.	Интеграция возможна, но требует дополнительных шаблонов, агентов и более ручной настройки под Kubernetes.
Язык запросов	PromQL — специализированный язык запросов для анализа метрик, агрегации и построения алертов.	Используются встроенные триггеры, элементы данных и выражения Zabbix; для Kubernetes-метрик менее удобен.
Визуализация	Grafana напрямую подключается к Prometheus и позволяет быстро строить наглядные дашборды.	Есть встроенные графики Zabbix; интеграция с Grafana возможна, но требует дополнительной настройки.
Развертывание и масштабирование	kube-prometheus-stack разворачивает Prometheus, Grafana, Alertmanager, kube-state-metrics и node-exporter через Helm.	Развертывание в Kubernetes возможно, но обычно сложнее и менее стандартизировано для cloud-native сценариев.

По результатам сравнения для проекта LabServer выбрана связка Prometheus + Grafana. Prometheus является cloud-native стандартом для Kubernetes-мониторинга: он хорошо работает с динамической природой кластера, поддерживает автоматическое обнаружение целей и легко разворачивается через Helm-чарты. В рамках лабораторной работы это особенно важно, так как стек мониторинга должен быть воспроизводимым на компьютерах обоих участников команды.

Для визуализации выбрана Grafana, поскольку она предоставляет удобный веб-интерфейс для построения дашбордов поверх данных Prometheus. С ее помощью можно отобразить состояние pod'ов, количество рестартов контейнеров и наличие проблемных состояний. В качестве источника Kubernetes-метрик используется kube-state-metrics,

позволяющий получать метрики жизненного цикла pod'ов, включая kube_pod_status_phase и kube_pod_container_status_restarts_total.

Практическая часть мониторинга была выполнена Студентом Б: в Minikube-кластере был развернут стек kube-prometheus-stack в пространстве имен monitoring, после чего в Grafana создан дашборд LabServer Kubernetes Monitoring. На дашборде отображаются количество работающих pod'ов проекта, статус backend, dashboard и postgres, количество рестартов контейнеров и число pod'ов в проблемных состояниях. Экспорт дашборда сохранен в репозитории в формате JSON для последующего импорта на другом компьютере.

4. Выводы

В ходе выполнения лабораторной работы был проведен комплексный анализ архитектуры многокомпонентного образовательного проекта LabServer. На основе полученных данных спроектирована отказоустойчивая инфраструктура для развертывания системы в среде оркестрации Kubernetes с использованием декларативных манифестов Deployment и Service.

Успешно решена задача обеспечения высокой доступности сервисов: разработаны и адаптированы правила автоматического мониторинга состояния контейнеров на уровне транспортного протокола с учетом временных задержек на инициализацию базы данных и компиляцию клиентских приложений. Для backend настроена tcpSocket-проверка порта 5288, для dashboard — tcpSocket-проверка порта 4200.

В рамках выбора инструментов мониторинга была обоснована связка Prometheus + Grafana. Prometheus выбран как cloud-native стандарт для Kubernetes, так как он использует pull-модель сбора метрик, поддерживает Kubernetes service discovery и разворачивается через Helm-чарты. Grafana выбрана для визуализации метрик и построения дашборда, удобного для демонстрации состояния сервисов.

Была выполнена практическая подготовка мониторинга: развернут kube-prometheus-stack в Minikube-кластере, проверен сбор метрик kube_pod_status_phase, создан Grafana-дэшборд LabServer Kubernetes Monitoring и экспортирован JSON-файл дашборда для воспроизводимого использования в репозитории. Полученная конфигурация позволяет отслеживать состояние pod'ов backend, dashboard и postgres, количество рестартов контейнеров и наличие проблемных состояний.

Итоговая архитектура обеспечивает как базовое самовосстановление средствами Kubernetes Liveness Probes, так и внешний сбор и визуализацию метрик средствами Prometheus и Grafana, что соответствует цели темы №17 по настройке развертывания служб проекта в рамках Kubernetes-кластера.

5. ПРИЛОЖЕНИЕ А. Листинги конфигурационных манифестов Kubernetes

Листинг 1 — Исходный код манифеста базы данных (db.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
      - name: postgres
        image: postgres:15
        env:
        - name: POSTGRES_PASSWORD
          value: "changeit"
        - name: POSTGRES_DB
          value: "labs"
        ports:
        - containerPort: 5432
---
apiVersion: v1
kind: Service
metadata:
  name: db-service
spec:
  selector:
    app: postgres
  ports:
  - port: 5432
```

Листинг 2 — Исходный код манифеста приложения (app.yaml)

```
# БЭКЕНД
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
      - name: backend
        image: trialextime/labserver-backend:v1
        ports:
        - containerPort: 5288
        env:
        - name: ConnectionStrings__PgSQL
          value: "Host=db-service;Port=5432;Database=labs;Username=postgres;Password=changeit"
        livenessProbe:
          tcpSocket:
            port: 5288
          initialDelaySeconds: 60
          periodSeconds: 15
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: backend
  ports:
  - port: 80
```

```
    targetPort: 5288

---
# ФРОНТЕНД - ДАШБОРД
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dashboard
spec:
  selector:
    matchLabels:
      app: dashboard
  template:
    metadata:
      labels:
        app: dashboard
    spec:
      containers:
      - name: dashboard
        image: trialextime/labserver-dashboard:v1
        ports:
        - containerPort: 4200
        livenessProbe:
          tcpSocket:
            port: 4200
          initialDelaySeconds: 120
          periodSeconds: 15
---
apiVersion: v1
kind: Service
metadata:
  name: dashboard-service
spec:
  selector:
    app: dashboard
  type: NodePort
  ports:
  - port: 80
    targetPort: 4200
    nodePort: 30000
```